Exercise: Anonymizing and Encrypting the Data

Explore how to anonymize sensitive credit card transaction data by masking numbers and encrypt account balances using symmetric key encryption. This lesson helps you apply data protection techniques within ETL transform steps using Python and pandas.

Data anonymization

Consider a dataset containing credit card transaction details of a bank's customers. The transactions are ATM deposits and withdrawals collected in real time as they happen. When a customer uses their credit card to withdraw or deposit money from an ATM, the transaction details are recorded in the dataset and stored in the bank's database.

Our job is to anonymize the dataset to protect the customer's credit card data. The dataset contains the following columns:

- `Customer ID`: A unique identifier for each customer
- `Transaction ID`: A unique identifier for each transaction
- `Credit Card Number`: The credit card number of the customer
- `Type`: The type of transactions; deposit or withdrawal
- `Amount`: The amount deposited or withdrawn in the transaction
- `Balance`: The balance after the transaction

Ask

Let's load a sample dataset into a pandas DataFrame:

main.py Python 3.10.4

```
import pandas as pd

# Loading credit card data
df = pd.DataFrame([
    ['1', '1', '123456789054', 'deposit', '100', '1533450'],
    ['1', '2', '123456789054', 'withdrawal', '50', '1533400'],
    ['2', '3', '234567890132', 'deposit', '200', '12424'],
    ['2', '4', '234567890132', 'withdrawal', '100', '12324'],
    ['3', '5', '345678901211', 'deposit', '300', '6534654']],
    columns=['Customer ID', 'Transaction ID', 'Credit Card Number', 'Type
)

# Print sample data
```

Run

Loading sample data to a pandas DataFrame

Task 1: Mask the credit card number

First, identify the column to anonymize. In this case, it is the `Credit Card Number`. To mask the credit card number, we need to replace the first 8 digits with asterisks.

main.py Python 3.10.4

```
# Grab the last 4 digits from the column "Credit Card Number"
last_4_digits = <<insertHere>>

# Generate a prefix of 8 asterisks
prefix = <<insertHere>>

# Modify the existing "Credit Card Number" column
<<insertHere>> = prefix + last_4_digits

# Print results
print(df.head())
```

Ask

Run

Anonymizing the credit card number column

Solution code: Anonymizing data

Here's the solution code:

main.py Python 3.10.4

```
# Grab the last 4 digits from the column "Credit Card Number"
last_4_digits = df["Credit Card Number"].str[-4:]

# Generate a prefix of 12 asterisks
prefix = 12 * '*'

# Modify the existing "Credit Card Number" column
df["Credit Card Number"] = prefix + last_4_digits

# Print results
print(df.head())
```

Run

Solution code: Anonymizing the credit card number column

- **Line 2:** We convert the column to a string and slice it to keep only the last 4 digits from each credit card number.
- **Line 5:** We generate a string of eight consecutive asterisks.
- **Line 8**: We modify the credit card numbers to concatenate the prefix and the last four digits.

Data encrypting

Ask

Next, we'd like to encrypt sensitive information such as the customer's account balance.

Task 2: Encrypt the account balance

In this case, use a symmetric key to encrypt the account balance column in the dataset.

main.py Python 3.10.4

```
from cryptography.fernet import Fernet

# Generating a new key
<<insertHere>>

# Encrypt sensitive data using a fernet key
def encrypt_data(df, key):
    f = Fernet(key)
    encrypted_data = {}

    # Convert the Balance column to bytes
    balance_column = df['Balance'].astype('str').to_numpy().tobytes()
```

Run

Encrypting the “Balance” column

Solution code: Encrypting data

Here’s the solution code:

main.py Python 3.10.4

```
from cryptography.fernet import Fernet

# Generating a new key
key = Fernet.generate_key()

# Encrypt sensitive data using the key
def encrypt_data(df, key):
    f = Fernet(key)
    encrypted_data = {}
```

Ask

```
# Convert the Balance column to bytes
balance_column = df['Balance'].astype('str').to_numpy().tobytes()
```

Run

Solution code: Encrypting the “Balance” column

- **Line 4:** We generate a key used to encrypt data in bytes format.
- **Line 12:** We convert the balance column to a stream of bytes.
- **Line 15:** We use the key to encrypt the data.
- **Line 19:** We apply the encryption function.
- **Line 20:** We use the function output to modify the current `Balance` column.
- **Line 23:** We print encrypted data.

Mark As Completed

Ask